

共享单车再平衡模型求解报告

深圳共享单车再平衡项目

2026 年 6 月 8 日

1 报告目的

共享单车再平衡模型本质上是带容量约束、装卸数量决策、车辆路径选择和短缺惩罚的混合整数规划问题。模型的核心形式可写为：

$$\begin{aligned} \min \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b, \\ & A_{\text{eq}}x = b_{\text{eq}}, \\ & \ell \leq x \leq u, \\ & x_i \in \mathbb{Z}, \quad i \in I. \end{aligned}$$

其中 x 包含路径变量、访问变量、服务停靠变量、装卸数量变量、车辆载重变量、未满足短缺变量和车辆完成时间变量。由于路径变量与访问变量大量为 0-1 变量，该问题不能直接用普通线性规划求解。

本项目实现了三类求解器：

1. 自写 MILP 精确求解器：两阶段修正单纯形法求 LP 松弛，分支定界处理整数变量。
2. 贪心插入 + 局部搜索 + 模拟退火启发式求解器：用于较大规模实例，快速得到可用调度方案。
3. 带启发式初始上界的 Branch-and-Cut 精确验证算法：用于小规模实例，验证启发式结果是否接近或达到最优。

三者定位不同：MILP 求解器追求精确性但效率较低；启发式求解器追求大规模可运行性；Branch-and-Cut 用于小规模精确验证，而不适合直接承担完整 200m 或 100m 大规模求解。

2 MILP 自写精确求解器

2.1 总体思路

自写 MILP 求解器位于 ‘src/model/solve.m’。其基本流程是：

1. 将当前分支节点下的 MILP 松弛成 LP。
2. 用两阶段修正单纯形法求解 LP 松弛。
3. 若 LP 不可行，则剪枝。
4. 若 LP 目标值已经不优于当前最好整数解，则剪枝。
5. 若 LP 解满足整数性，则更新当前最好解。
6. 若存在分数整数变量，则选择一个变量分支，生成两个子问题。

核心分支定界结构可概括为：

Listing 1: MILP 分支定界主循环

```
1 root = makeNode mdl.lb, mdl.ub, 0);
2 stack = root;
3 bestX = [];
4 bestObj = inf;
5
6 while ~isempty(stack)
7     node = stack(end);
8     stack(end) = [];
9
10    lp = solveLpRelaxation(mdl, node.lb, node.ub, opts, tStart);
11    if lp.status == "infeasible"
12        continue;
13    end
14    if lp.obj >= bestObj - opts.tol
15        continue;
16    end
17
18    [isInt, branchVar] = checkIntegrity(lp.x, mdl.intcon, ...
19        opts.integer_tol, opts.branch_rule);
20
21    if isInt
22        bestX = lp.x;
```

```

23     bestObj = lp.obj;
24     else
25         left.ub(branchVar) = floor(lp.x(branchVar));
26         right.lb(branchVar) = ceil(lp.x(branchVar));
27         stack = [stack, left, right];
28     end
29 end

```

2.2 LP 松弛标准型转换

每个分支节点会修改变量上下界。设当前节点给出的上下界为 ℓ' 和 u' 。求解器先做变量平移：

$$x = \ell' + z, \quad z \geq 0.$$

原约束变为：

$$Az \leq b - A\ell',$$

$$A_{\text{eq}}z = b_{\text{eq}} - A_{\text{eq}}\ell'.$$

对有限上界 u'_j 还需加入：

$$z_j \leq u'_j - \ell'_j.$$

随后给不等式加入松弛变量 $s \geq 0$ ，形成标准型：

$$\begin{aligned}
 & \min \quad \tilde{c}^\top z \\
 & \text{s.t.} \quad \begin{bmatrix} A_{\text{ineq}} & I \\ A_{\text{eq}} & 0 \end{bmatrix} \begin{bmatrix} z \\ s \end{bmatrix} = \begin{bmatrix} b_{\text{ineq}} \\ b_{\text{eq}} \end{bmatrix}, \\
 & \quad z \geq 0, \quad s \geq 0.
 \end{aligned}$$

核心代码结构为：

Listing 2: LP 标准型构造

```

1  mapBack = find(ub > lb);
2  shift = lb;
3  c = mdl.c(mapBack);
4  baseObj = mdl.c' * shift;
5
6  Aineq = mdl.Aineq(:, mapBack);
7  bineq = mdl.bineq - mdl.Aineq * shift;
8  Aeq = mdl.Aeq(:, mapBack);
9  beq = mdl.beq - mdl.Aeq * shift;
10

```

```

11 finiteUb = isfinite(ub(mapBack));
12 ubRows = speye(numel(mapBack));
13 ubRows = ubRows(finiteUb, :);
14 ubRhs = ub(mapBack(finiteUb)) - lb(mapBack(finiteUb));
15
16 Aineq = [Aineq; ubRows];
17 bineq = [bineq; ubRhs];
18 A = [Aineq, speye(mIneq); Aeq, sparse(mEq, mIneq)];

```

2.3 两阶段修正单纯形法

由于等式约束和部分约束不一定天然给出一个可行初始基，求解器采用两阶段法。第一阶段引入人工变量 $a \geq 0$ ，求解：

$$\min \quad \mathbf{1}^\top a$$

$$Ax + I_a a = b, \quad x \geq 0, a \geq 0.$$

若第一阶段最优值大于 0，则原 LP 不可行；若等于 0，则得到一个原问题可行基，再进入第二阶段优化原目标。

在修正单纯形法中，设当前基矩阵为 $B = A_B$ ，基变量为 x_B 。每轮迭代计算：

$$x_B = B^{-1}b,$$

$$y = B^{-\top} c_B,$$

$$\bar{c}_j = c_j - a_j^\top y.$$

若所有非基变量的约化成本 $\bar{c}_j \geq 0$ ，则当前解为 LP 最优解。否则选择 $\bar{c}_j < 0$ 的变量入基，并计算方向：

$$d = B^{-1}a_j.$$

离基变量由比值检验确定：

$$\theta = \min_{i: d_i > 0} \frac{x_{B_i}}{d_i}.$$

核心代码结构为：

Listing 3: 修正单纯形法核心迭代

```

1 B = A(:, basis);
2 xB = basisSolve(B, b);
3
4 y = basisSolve(B', c(basis));
5 rc = c - A' * y;
6 rc(basis) = 0;

```

```

7
8 [minRc, entering] = min(rc);
9 if minRc >= -tol
10     x = zeros(n, 1);
11     x(basis) = max(0, xB);
12     return;
13 end
14
15 d = basisSolve(B, A(:, entering));
16 positive = d > tol;
17 ratio = inf(m, 1);
18 ratio(positive) = xB(positive) ./ d(positive);
19 [~, leavingRow] = min(ratio);
20 basis(leavingRow) = entering;

```

2.4 分支策略

若 LP 解中某个整数变量 x_j 不是整数，则分为两个子问题：

$$x_j \leq \lfloor x_j^* \rfloor,$$

$$x_j \geq \lceil x_j^* \rceil.$$

当前实现优先选择最接近 0.5 的分数变量：

$$j^* = \arg \min_{j \in I, x_j \notin \mathbb{Z}} |(x_j - \lfloor x_j \rfloor) - 0.5|.$$

这样做的直觉是：越接近 0.5 的变量不确定性越强，优先分支有助于更快切开搜索空间。

2.5 效率低下的原因

虽然该求解器能够完整体现 MILP 精确求解思想，但在本项目规模下效率明显低于成熟商业求解器。主要原因如下。

2.5.1 问题本身是 NP-hard

共享单车再平衡模型包含车辆路径问题的结构。即使不考虑装卸数量变量和短缺惩罚，容量车辆路径问题本身也是 NP-hard。分支定界最坏情况下需要枚举指数级节点。若有 m 个 0-1 变量，理论搜索空间可达：

$$O(2^m).$$

在模型中, 路径变量规模约为:

$$|E| \cdot |K|,$$

访问变量和服务变量规模约为:

$$|N| \cdot |K|.$$

当网格从 1km 变为 200m 或 100m, $|N|$ 和 $|E|$ 都会显著增长。

2.5.2 每个分支节点都重新求 LP

当前实现每个分支节点都会重新构造 LP 标准型, 并从两阶段单纯形法开始求解。成熟求解器通常会大量使用 warm start、对偶单纯形、基继承、预处理和节点间信息复用。当前自写求解器的流程更接近:

每个节点 $\Rightarrow$ 重新标准化 $\Rightarrow$ 重新 Phase I $\Rightarrow$ 重新 Phase II.

这会造成大量重复计算。

2.5.3 数值退化和 Big-M 约束影响效率

载重转移和服务启用约束中存在 Big-M 结构。即使已经采用车辆相关的

$$M_k = 2Q_k + \max_i s_i + \max_j r_j,$$

LP 松弛中仍会出现退化基、接近奇异的基矩阵以及较弱的下界。退化会导致单纯形多次换基但目标值几乎不变, 从而拖慢 LP 松弛求解。

2.5.4 MTZ 与节点拆分增加模型规模

MTZ 子回路消除需要访问顺序变量 o_{ik} , 并对服务边加入:

$$o_{ik} - o_{jk} + |N|x_{ijk} \leq |N| - 1.$$

若使用节点拆分, 真实网格 g 被拆为多个访问副本 $V(g)$, 则有效节点数量从原来的 $|N_0|$ 增加为:

$$|N| = \sum_{g \in N_0} |V(g)|.$$

这使路径变量、访问变量、载重变量和 MTZ 约束同步增加。

2.5.5 缺少成熟求解器的全套加速机制

当前求解器没有实现或只实现了较简单版本的下列机制:

- 强预处理与变量固定;
- 强分支、伪成本分支;

- 割平面族，如 Gomory 割、容量割、连通性割；
- 启发式整数可行解搜索；
- LP warm start 与对偶单纯形；
- 稀疏 LU 分解更新。

因此，MILP 求解器更适合作为课程项目中展示精确求解原理和小规模验证的工具，而不适合作为大规模主求解器。

3 贪心插入 + 局部搜索 + 模拟退火求解器

3.1 总体思路

启发式求解器位于 ‘src/model/solve_heuristic.m’。它不试图证明全局最优，而是围绕原目标函数快速构造高质量可行解。其基本思想是：

1. 在稀疏邻接图上计算所有节点间最短路。
2. 将一次调度抽象为任务 (i, j, w) ：从富余节点 i 装 w 辆车，到短缺节点 j 卸 w 辆车。
3. 用贪心插入逐步生成初始任务序列。
4. 用 relocate、swap、reverse、intra-relocate 等邻域操作产生新解。
5. 用模拟退火准则接受或拒绝新解。

启发式状态可表示为：

$$R_k = [(i_1, j_1, w_1), (i_2, j_2, w_2), \dots],$$

其中 R_k 是车辆 k 的任务序列。每个任务表示先访问富余副本 i ，再访问短缺副本 j 。

3.2 稀疏图最短路

车辆实际行驶路径不一定直接从任务点到任务点，而是在稀疏邻接图 $G = (N \cup \{0\}, E)$ 上走最短路径。启发式首先计算：

$$\delta_{ij} = \text{dist}_G(i, j),$$

其中 δ_{ij} 是稀疏图上从 i 到 j 的最短路距离。当前实现使用 Floyd-Warshall 型三重循环：

$$\delta_{ij} \leftarrow \min(\delta_{ij}, \delta_{ik} + \delta_{kj}).$$

核心代码为：

Listing 4: 稀疏图全源最短路

```

1  for e = 1:data.edge_count
2      i = data.edge_from(e);
3      j = data.edge_to(e);
4      dist(i, j) = data.edge_dist_km(e);
5      nextNode(i, j) = j;
6  end
7
8  for k = 1:nNode
9      for i = 1:nNode
10         for j = 1:nNode
11             alt = dist(i, k) + dist(k, j);
12             if alt < dist(i, j)
13                 dist(i, j) = alt;
14                 nextNode(i, j) = nextNode(i, k);
15             end
16         end
17     end
18 end

```

3.3 路线时间计算

对车辆 k ，若其任务序列为：

$$R_k = \{(i_t, j_t, w_t)\}_{t=1}^{m_k},$$

则路线时间可近似写为：

$$T_k = \sum_{t=1}^{m_k} \left[\frac{60\delta_{\text{cur}_t, i_t}}{v_k} c_k + a_k w_t \frac{60\delta_{i_t, j_t}}{v_k} c_k + b_k w_t \right] + \frac{60\delta_{\text{last}, 0}}{v_k}.$$

其中 a_k 和 b_k 分别为车辆 k 的单位装车、卸车时间， c_k 为固定停靠服务时间。

核心代码为：

Listing 5: 启发式路线时间计算

```

1  t = t + 60 * spDist(current, pickup) / data.speed(kk);
2  t = t + data.fixed_stop_time(kk) + data.load_time(kk) * qty;
3  t = t + 60 * spDist(pickup, dropoff) / data.speed(kk);
4  t = t + data.fixed_stop_time(kk) + data.unload_time(kk) * qty;
5  current = dropoff;
6
7  if returnToDepot && current ~= 1

```

```

8   t = t + 60 * spDist(current, 1) / data.speed(kk);
9   end

```

3.4 评价函数

启发式使用与 MILP 基本一致的复合目标：

$$F(R) = \alpha \max_k T_k \gamma \sum_{k \in K} T_k \sum_{j \in D} \beta_j u_j + P_{\text{service}}.$$

若未达到最低服务水平，则加入软惩罚：

$$P_{\text{service}} = \lambda \max \left(0, \theta \min \left(\sum_j r_j, \sum_i s_i \right) - \sum_j \text{served}_j \right).$$

当前代码中 λ 取较大值：

$$\lambda = 100 \max_j \beta_j.$$

这样启发式不会因为短期找不到硬可行解而崩溃，但会强烈倾向于提高服务水平。

核心代码为：

Listing 6: 启发式目标函数

```

1   served = sum(state.served);
2   requiredServed = cfg.service_level_required * ...
3       min(sum(data.shortage), sum(data.surplus));
4   serviceShortfall = max(0, requiredServed - served);
5
6   unmetPenalty = data.beta' * state.unmet;
7   makespan = max(vehicleTime);
8   totalTime = sum(vehicleTime);
9
10  obj = cfg.alpha_makespan * makespan ...
11      + cfg.gamma_total_time * totalTime ...
12      + unmetPenalty;
13
14  if cfg.include_service_level_constraint && serviceShortfall > 0
15      servicePenalty = 100 * max(data.beta) * serviceShortfall;
16      obj = obj + servicePenalty;
17  end

```

3.5 贪心插入

贪心阶段枚举候选车辆、富余节点和短缺节点，令一次任务的调度量为：

$$w = \min\{S_i^{\text{remain}}, R_j^{\text{remain}}, Q_k\}.$$

对于每个候选任务，将其插入当前路线末尾并重新评价目标函数，选择目标函数改善最大的候选。用公式表示，当前状态为 R ，候选动作集合为 $\mathcal{M}(R)$ ，则选择：

$$m^* = \arg \min_{m \in \mathcal{M}(R)} F(R \oplus m).$$

核心代码为：

Listing 7: 贪心插入核心逻辑

```

1 for kk = vehicleOrder
2     for si = surplusNodes
3         for dj = shortageNodes
4             qty = min([remaining_surplus(si), ...
5                         remaining_shortage(dj), ...
6                         data.capacity(kk)]);
7             trial = state;
8             trial.routes{kk} = [trial.routes{kk}; si, dj, qty];
9             trial.remaining_surplus(si) = trial.remaining_surplus(si) - qty;
10            trial.remaining_shortage(dj) = trial.remaining_shortage(dj) - qty;
11            trialObj = evaluateState(trial, data, cfg, spDist);
12            if trialObj < bestObj
13                bestMove = trial;
14            end
15        end
16    end
17 end

```

3.6 局部搜索邻域

贪心解通常较快但容易陷入局部结构。当前启发式使用四类邻域：

邻域	含义
relocate	将一辆车中的某个任务移动到另一辆车或同一辆车的其他位置
swap	交换两辆车或同一辆车中的两个任务
reverse	反转某辆车任务序列中的一段顺序

邻域	含义
intra-relocate	在同一辆车内部移动某个任务的位置

这些操作改变车辆任务分配和访问顺序，但不改变每个任务的装卸数量，因此能保持需求和供给更新逻辑相对简单。

3.7 模拟退火接受准则

设当前解目标为 $F(R)$ ，候选解目标为 $F(R')$ ，差值为：

$$\Delta = F(R') - F(R).$$

若 $\Delta \leq 0$ ，直接接受；若 $\Delta > 0$ ，以概率接受：

$$P(\text{accept}) = \exp\left(-\frac{\Delta}{\tau}\right),$$

其中 τ 为当前温度。温度按几何降温：

$$\tau_{t+1} = \max(\tau_{\min}, \rho\tau_t).$$

核心代码为：

Listing 8: 模拟退火接受准则

```

1 candidate = makeNeighbor(current, data);
2 [candObj, candParts] = evaluateState(candidate, data, cfg, spDist);
3 delta = candObj - currentObj;
4
5 if isfinite(candObj) && ...
6     (delta <= 0 || rand() < exp(-delta / max(temperature, min_temperature)))
7     current = candidate;
8     currentObj = candObj;
9 end
10
11 if currentObj < bestObj
12     best = current;
13     bestObj = currentObj;
14 end
15
16 temperature = max(min_temperature, temperature * cooling_rate);

```

3.8 适用场景

该启发式求解器适合 1km、500m、200m 甚至 100m 这类较大规模实例，尤其适合需要快速获得可视化路线和可执行调度方案的情形。它的主要优点是：

- 不依赖 MATLAB 优化工具箱；
- 能直接生成车辆路线和装卸动作；
- 计算时间可通过迭代次数、候选采样数和服务节点筛选控制；
- 可以自然处理节点拆分后的访问副本。

缺点是无法证明全局最优，结果质量依赖参数、邻域设计和初始解质量。

4 带启发式初始上界的 Branch-and-Cut 精确验证

4.1 设计目的

Branch-and-Cut 求解器位于 ‘src/model/solve_branch_cut.m’。它不是用于直接求解完整大规模模型，而是用于小规模精确验证。其核心思想是：

1. 先用启发式求得一个可行解，作为当前上界 UB 。
2. 加入目标割，要求精确求解器寻找比当前上界更优的解。
3. 若找不到更优整数解，则证明当前启发式解在验证模型中最优。
4. 若找到整数解，则检查是否存在不连接调度中心的子回路。
5. 若存在子回路，则加入子回路割并继续求解。

4.2 启发式初始上界

若启发式解 x^H 可行，其目标值为：

$$UB = c^\top x^H.$$

为了验证是否存在更优解，加入目标割：

$$c^\top x \leq UB - \varepsilon.$$

若加入该割后模型无整数可行解，则说明不存在严格优于 x^H 的整数解，因此 x^H 可被证明为当前验证模型的最优解。

核心代码为：

Listing 9: 使用启发式解作为上界

```

1 heuristicFeasible = heuristicSol.status == "heuristic_feasible";
2 if heuristicFeasible && isfinite(heuristicSol.objective)
3     bc.incumbent_objective = heuristicSol.objective;
4 else
5     bc.incumbent_objective = inf;
6 end
7
8 if isfinite(bc.incumbent_objective)
9     work = addObjectiveCut(work, ...
10         bc.incumbent_objective - cfg.exact_validation.objective_epsilon);
11 end

```

目标割实现为:

Listing 10: 目标割 $c'x \leq UB - \epsilon$

```

1 function mdl = addObjectiveCut(mdl, upperBound)
2     mdl.Aineq = [mdl.Aineq; sparse(1, 1:numel(mdl.c), ...
3         mdl.c(:)', 1, numel(mdl.c))];
4     mdl.bineq = [mdl.bineq; upperBound];
5 end

```

4.3 子回路检测

对每辆车 k , 从整数解中取出被使用的边:

$$E_k^+ = \{(i, j) \in E : x_{ijk} = 1\}.$$

若这些边形成的连通分量中存在一个分量 C 不包含调度中心 0, 且 $|C| \geq 2$, 则该分量是非法子回路。当前实现将有向边临时看作无向边, 寻找不含 depot 的连通分量。

核心代码为:

Listing 11: 整数解中的子回路检测

```

1 usedEdges = find(x(idx.x(:, kk)) > 0.5);
2 graphEdges = [data.edge_from(usedEdges), data.edge_to(usedEdges)];
3
4 components = connectedComponentsUndirected(graphEdges, data.n + 1);
5 for c = 1:numel(components)
6     comp = components{c};
7     if any(comp == 1)
8         continue; % contains depot

```

```

9     end
10    internalServiceNodes = comp(comp > 1) - 1;
11    if numel(internalServiceNodes) >= 2
12        cuts(end+1) = struct("vehicle", kk, "nodes", internalServiceNodes);
13    end
14 end

```

4.4 子回路割

若发现某辆车 k 在服务节点集合 S 上形成不含调度中心的子回路，则加入 SEC 割：

$$\sum_{i \in S} \sum_{j \in S} x_{ijk} \leq |S| - 1.$$

该割的含义是：集合 S 内部不能形成一个完整闭环，至少要有一条连接到集合外部的边，才能与调度中心所在路径连通。

Listing 12: 子回路割 SEC

```

1 function mdl = addSubtourCut(mdl, vehicle, nodes)
2     edgeMask = ismember(data.edge_from - 1, nodes) & ...
3         ismember(data.edge_to - 1, nodes);
4     edgeVars = idx.x(find(edgeMask), vehicle);
5     row = sparse(1, edgeVars, 1, 1, numel(mdl.c));
6     mdl.Aineq = [mdl.Aineq; row];
7     mdl.bineq = [mdl.bineq; numel(nodes) - 1];
8 end

```

4.5 Branch-and-Cut 主流程

整体流程可写为：

Listing 13: Branch-and-Cut 精确验证流程

```

1 work = mdl;
2 if heuristic solution is feasible
3     work = addObjectiveCut(work, heuristicObj - epsilon);
4 end
5
6 for roundId = 1:maxCutRounds
7     exactSol = solve(work, localCfg);
8
9     if exactSol.status == "no_integer_solution"

```

```
10     prove heuristic incumbent optimal;
11     break;
12 end
13
14 cuts = findSubtourCuts(exactSol.x, work);
15 if ~isempty(cuts)
16     for each cut
17         work = addSubtourCut(work, cut.vehicle, cut.nodes);
18     end
19     continue;
20 end
21
22 record exact solution;
23 end
```

4.6 为什么它只适合小规模验证

Branch-and-Cut 看起来比一次性加入 MTZ 更灵活，但在当前自写求解器环境下，它仍然可能较慢：

- 每一轮加割后都要重新调用内部 MILP 求解器；
- 内部 MILP 求解器本身没有 warm start；
- 若目标割较紧，证明“没有更优解”往往比找到一个可行解更难；
- 若模型规模较大，每轮 LP 松弛已经很慢；
- 子回路割可能需要多轮才能完全排除所有非法环。

因此，本文将 Branch-and-Cut 定位为“精确验证算法”，而不是大规模主求解器。推荐使用方式是：

- 大规模模型：使用贪心插入 + 局部搜索 + 模拟退火；
- 小规模抽样模型：使用 Branch-and-Cut 验证启发式误差；
- 结果报告：同时展示启发式目标值、验证模型精确目标值和 gap。

5 三种求解器对比

求解器	优点	缺点	适用场景
自写 MILP 分支定界	原理完整，可证明小规模最优	LP 反复求解，效率低，规模稍大即困难	教学展示、小规模模型
贪心插入 + 局部搜索 + 模拟退火	快速，可生成实际路线，可用于大规模	不能证明最优，依赖参数与初始解	主求解器、200m/100m 大规模实验
带启发式上界的 Branch-and-Cut	可验证启发式解是否最优或接近最优	多轮 MILP，效率仍低	小规模精确验证、算法对比

6 结论

本项目的三类求解器形成了互补关系。MILP 求解器展示了从 LP 松弛、两阶段修正单纯形到分支定界的完整精确求解链路，但由于共享单车再平衡模型具有车辆路径问题的组合爆炸特征，且自写求解器缺少成熟商业求解器的预处理、割平面、warm start 和高级分支策略，因此求解效率较低。

贪心插入 + 局部搜索 + 模拟退火求解器是当前最适合作为主求解器的方法。它通过任务化建模、稀疏图最短路、软服务水平惩罚和模拟退火邻域搜索，在较短时间内生成可执行调度方案。

Branch-and-Cut 精确验证算法则用于小规模实例。它利用启发式解作为初始上界，通过目标割搜索更优解，并用子回路割排除不连通的非法路径。当加入目标割后内部求解器返回 ‘no_integer_solution’ 时，其含义是“不存在比当前启发式上界更优的整数解”，因此可以证明启发式解在该验证模型中最优。

因此，建议在报告和实验中采用如下求解策略：

大规模求解使用启发式， 小规模样本使用 Branch-and-Cut 做精确验证。